

Set up Stripe payments

A walkthrough for DL Coaching. Customers will buy a single session or a package through Stripe Checkout. Once Stripe confirms payment to the website, the customer receives session credits and can book available times.

Start with a sandbox. It lets you test without moving real money. Live payments stay off until the developer has verified payment, credits and booking together. [S1]

Who does what

Darryl	Developer
Owns the Stripe account; enters business, identity and bank details directly into Stripe.	Connects the website, configures payment notifications and checks the full test flow.
Approves the public business details, prices and cancellation/refund policy.	Stores secrets, switches the approved deployment to live mode and verifies configuration.

1. Create or open your business account

Open [Stripe registration](#), or sign in to your existing account. Use an email you control. Complete Stripe's account setup prompts with accurate business information and the correct country and business type. [S2]

Have the business address, support contact, business description, payout bank information and any identity documents Stripe requests ready. Enter these yourself in Stripe; you do not need to send them to the developer. Complete any outstanding activation requirements before accepting real payments. [S2]

2. Check what customers will see

Review the business name, website, support details and statement descriptor in Stripe. Use details that customers will recognise as DL Coaching. Review receipt settings and confirm the business policies that will be published on the website. [S2]

The prices this website uses

Offer	Sessions	One-off price
Single session	1	£45
Starter	4	£150
Transformation	8	£250
Complete Transformation	16	£450

These are one-off GBP payments, not subscriptions. The current website creates the Checkout line items itself. You do not need to create Payment Links, recurring plans or separate Stripe products manually.

Connect the test environment

3. Open a Stripe sandbox

In Stripe's account picker, choose **Sandboxes**, then create or open a sandbox for DL Coaching. Stay inside that sandbox for both the key and webhook setup. Existing integrations may use test mode instead; do not mix keys and events from different environments. [S1]

4. Store the secret API key

Open the sandbox's **API keys** page (search for API keys if the menu differs). Copy its secret key, normally starting **sk_test_**, to the agreed secure secret store. The developer will set **STRIPE_SECRET_KEY**. This website does not need a publishable key for its hosted Checkout redirect. [S3]

Keep secret keys out of WhatsApp, ordinary email, source code and screenshots. Use a password-manager secure share or enter them directly in the deployment environment with the developer. Darryl keeps ownership of the Stripe account.

5. Create the payment notification destination

A webhook is Stripe's message to the website confirming payment. The developer must first provide a deployed HTTPS address that can receive Stripe requests. Do not use the local preview address.

In **Workbench > Webhooks**, choose **Create an event destination**. Select **Your account**. For this handler, use snapshot events and an API version agreed with the developer. Select these two events only: [S4]

```
checkout.session.completed  
checkout.session.async_payment_succeeded
```

Choose **Continue > Webhook endpoint**. Enter the confirmed website address followed by the path below, then create the destination. Replace YOUR-CONFIRMED-DOMAIN; it is a placeholder. [S4]

```
https://YOUR-CONFIRMED-DOMAIN/api/stripe-webhook
```

Open the destination, reveal its signing secret, and securely store the **whsec_...** value as **STRIPE_WEBHOOK_SECRET**. This is a different secret from the API key. [S4]

Checkpoint: the test API key, webhook and signing secret all belong to the same Stripe sandbox or test environment. The developer has deployed the settings before you try checkout.

Test, then go live

6. Make a test purchase through the website

- The developer provides the test website, connected to a separate test database/Auth environment. Test purchases must not create spendable credits in the live customer database.
- Create a test customer account and choose Starter. Confirm Stripe displays £150 and the correct offer. Check that you are in the sandbox/test environment.
- Use the test card below. After payment, return to the website and refresh if needed. Your balance should increase by exactly four credits.
- Repeat for the other offers: one, eight and sixteen credits respectively. The developer checks successful webhook delivery and confirms that replaying a notification does not add credits again.

Test card: 4242 4242 4242 4242
Expiry: any future date | CVC: any three digits

Use Stripe test cards only in the sandbox/test environment. Also test a decline using **4000 0000 0000 0002**; no credits should be awarded. [S5]

Finish by booking a published slot with a test credit and confirming the Google Calendar event and invitation. Tell any test recipient in advance. Payment success by itself does not prove booking works.

7. Switch to live only after the checks pass

- Darryl completes Stripe activation, payout-bank setup and any outstanding verification. Confirm support details, business policies and approved prices.
- In the live Stripe account, create or obtain the live secret key (normally sk_live_). Store it securely when shown; newly created live secrets may only be displayed once. [S3]
- Create the live webhook destination for the confirmed production domain, with the same two events. Store its own whsec_ signing secret; the test secret cannot be reused.
- The developer sets the live keys only in Production, sets PUBLIC_SITE_URL to the exact live HTTPS origin, confirms the live database/Auth settings, then redeploys.
- Darryl and the developer confirm the launch checklist is complete. Keep Preview and local development on test credentials and a test database.

Refunds need coordination. The current website does not automatically remove credits or cancel bookings when a payment is refunded in Stripe. Darryl must coordinate the refund and account correction with the developer.

Developer handoff & help

Required server settings

Setting	What belongs here
STRIPE_SECRET_KEY	The secret API key for the selected test or live environment.
STRIPE_WEBHOOK_SECRET	The signing secret for that environment's exact webhook endpoint.
PUBLIC_SITE_URL	The confirmed HTTPS website origin, without a page path. Used for checkout return URLs.

Add settings to the intended hosting project and redeploy. Keep secrets server-side; never use a VITE_ prefix. Neon database/Auth must already work. For local API testing use **vercel dev**; the Vite preview alone cannot run the payment endpoints.

Implementation checks specific to this website

- Use the website's actual Checkout flow to test fulfillment. A generic test event may lack the user and package metadata the handler requires.
- The current Stripe SDK uses API version 2026-01-28.clover. Match the webhook version where available, or verify compatibility before choosing another version.
- The webhook validates GBP and the exact package amount. Do not add discounts or tax on top of these amounts without updating and testing the payment validation.
- A payment-return URL alone never awards credits. Only a signed, paid Checkout event can do that. A delayed webhook may require a page refresh.

Common problems

What you see	What to check
Checkout will not open	Customer sign-in, deployed API routes, STRIPE_SECRET_KEY and PUBLIC_SITE_URL.
Payment made; no credits	Workbench > Webhooks > Event deliveries, signing secret, metadata and database errors. Do not ask the customer to pay again.
Webhook returns 400	Correct endpoint secret and raw-body signature verification; check test/live environment pairing.
Webhook returns 401/403/404	Confirm the URL, route deployment and hosting protection. Stripe must reach this route; retain signature verification.

Official help

[S1 - Sandboxes and environment separation](#)

[S2 - Business account setup](#)

[S3 - API keys and secure storage](#)

[S4 - Webhook setup and delivery logs](#)

[S5 - Test cards and declines](#)